

Chapter 23 - Semantic Reasoning: Stage 6 of the Semantic Knowledge Development Lifecycle

Deriving New Knowledge Through Automated Inference

The first five stages of the Semantic Knowledge Development Lifecycle (SKDL) transformed an abstract conceptual vocabulary into a **governed, reusable, validated** knowledge asset.

- Conceptual Modeling gave us structure.
- Semantic Description gave us meaning.
- Knowledge Reuse gave us composability.
- Semantic Governance gave us discipline.
- Semantic Validation gave us trust.

Now, as knowledge begins to flow from ontology models into production systems, a new challenge emerges:

"What new knowledge can we derive from what we already know?"

Semantic Reasoning answers this question.

It is the discipline of deriving new, logical entailed knowledge from an existing ontology through automated inference.

It transforms a static knowledge base into a dynamic, intelligent system capable of discovering relationships, classifications, and consequences that were never explicitly asserted.

Reasoning is the engine of intelligence in an ontology.

This chapter examines **Exercise 20** and **Exercise 21** from Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial* through the lens of semantic reasoning. While **Exercise 18** introduced governance mechanisms (disjointness axioms), and **Exercise 19** introduced validation (the Probe Class patter), **Exercise 20** and **Exercise 21** introduce the core distinction that makes reasoning possible:

- **Primitive Classes** (defined with `SubClassOf`) express only necessary conditions. They describe what something *must* be, but they do not enable auto-classification.
- **Defined Classes** (defined with `EquivalentTo`) express both necessary and sufficient conditions. They describe what something *is*, and they also enable the reasoner to classify individuals and classes automatically.

This distinction is the **gateway to automated reasoning**. Once a class is defined with sufficient conditions, the reasoner can recognize instances of that class without being told.

Along the way, we will examine reasoning from multiple complementary perspectives: mathematical, historical, engineering, architectural, and industrial -- revealing that

automated reasoning is not a modern invention, but the culmination of more than two thousand years of logical thought.

Table of Content

- [23.1 Chapter Introduction -- From Verification to Discovery](#)
 - [23.1.1 The Journey So Far](#)
 - [23.1.2 The New Question](#)
 - [23.1.3 The Central Insight](#)
- [23.2 Learning Objectives](#)
- [23.3 Positioning Within SKDL - Stage 6 Highlighted](#)
- [23.4 What Is Semantic Reasoning?](#)
 - [23.4.1 A Working Definition](#)
 - [23.4.2 Asserted Knowledge vs. Inferred Knowledge](#)
 - [23.4.3 Reasoning vs. Querying vs. Validation](#)

- [23.5 Exercise 20 -- Primitive Classes and the Limits of Inference](#)
 - [23.5.1 The Engineering Objective](#)
 - [23.5.2 Implementation: Creating the Primitive Class](#)
 - [23.5.3 The Result: What the Reasoner Does Not Do](#)
 - [23.5.4 Why Primitive Classes DO Not Enable Auto-Classification](#)
 - [23.5.5 The Engineering Value of Primitive Classes](#)
 - [23.5.6 Engineering Perspective: Exercise 20 as a Case Study](#)
- [23.6 Exercise 21 -- Defined Classes and the Power of Auto-Classification](#)
 - [23.6.1 The Engineering Objective](#)
 - [23.6.2 Implementation: Converting to a Defined Class](#)
 - [23.6.3 Why Defined Classes Enable Auto-Classification](#)
 - [23.6.4 The Moment of Auto-Classification](#)
 - [23.6.5 Primitive vs. Defined Classes: A Summary](#)
 - [23.6.6 Engineering Perspective: Exercise 21 as a Case Study](#)

23.1 Chapter Introduction -- From Verification to Discovery

23.1.1 The Journey So Far

The first five stages of the Semantic Knowledge Development Lifecycle (SKDL) established the foundations of ontology engineering.

Stage 1 -- Conceptual Modeling identified the concepts that define a knowledge domain and organized them into a coherent semantic taxonomy.

Stage 2 -- Semantic Description enriched those concepts with formal meaning using Description Logic, OWL restrictions, and class expressions.

Stage 3 -- Knowledge Reuse transformed validated semantic knowledge into reusable assets that could be shared, extended, and specialized systematically.

Stage 4 -- Semantic Governance established the policies, standards, and processes required to maintain semantic integrity across an evolving knowledge ecosystem.

Stage 5 -- Semantic Validation verified that the ontology is logically consistent, semantically complete, and fit for purpose before deployment.

By the end of Stage 5, the `Pizza` ontology had been validated. It was consistent, satisfiable, and free of contradictions. The governance rules had been enforced. The probe class had been created, used, and removed.

The ontology was correct.

But correctness is not the same as intelligence.

A correct ontology that cannot derive new knowledge is like a well-organized library where no one can find a needed book. It contains knowledge, but it does not *use* knowledge. It is passive, not active.

This is where **Stage 6 -- Semantic Reasoning** enters the lifecycle.

23.1.2 The New Question

Validation asked: **"Is this model broken?"**

Reasoning asks: **"What else can this model tell us?"**

The distinction is fundamental.

Validation ensures that the ontology is logically sound. Reasoning derives new knowledge from that sound foundation.

Without validation, reasoning produces unsound inferences.

Without reasoning, validation produces a correct but static model.

Together, they produce trustworthy intelligence.

23.1.3 The Central Insight

The central insight of this chapter is deceptively simple:

Reasoning does not generate new facts -- it makes explicit what was already implicit in the axioms.

The reasoner does not invent knowledge. It discovers the logical consequences of the axioms the engineer has written. Every inferred statement was already entailed by the ontology. The reasoner simply makes the entailment explicit.

This is the power of formal semantic:

from a seed of **asserted knowledge**, the reasoner grows a tree of **inferred knowledge**.

23.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

Knowledge:

- Explain why Semantic Reasoning is the sixty stage of the Semantic Knowledge Development Lifecycle (SKDL)
- Distinguish between **primitive classes** (`SubClassOf` only) and **defined classes** (`EquivalentTo`)
- Understand the difference between **asserted knowledge** and **inferred knowledge**
- Explain how a reasoner derives new subclass relationships automatically
- Identify major RDF reasoners beyond Protégé and explain their differences
- Understand how reasoning applies in industrial contexts (problem-solving, decision-making, safety, compliance)
- Explain the mathematical foundations of reasoning: subsumption, entailment, tableau algorithm, decidability

Practical Skills:

- Create a primitive class (`CheesyPizza`) using `SubClassOf` only
- Convert a primitive class to a defined class using `Edit > Convert to defined class`
- Observe the reasoner automatically classifying individuals and classes
- Use Protégé's explanation feature to understand why an inference was made
- Compare reasoning results across different reasoners (Pellet, HermiT, ELK, JFact)
- Select the appropriate reasoner for a given task

Engineering Perspective:

- Understand reasoning as "automated knowledge discovery"
- Explain why defined classes enable automation while primitive classes do not
- Map reasoning activities to the EKA R layer
- Apply reasoning to industrial problem-solving, safety, and compliance scenarios

Mathematical Perspective:

- Understand model-theoretic semantics: interpretations, models, entailment
- Explain subsumption as set inclusion and logical consequence
- Understand the tableau algorithm as a decision procedure
- Explain decidability and computational complexity
- Connect reasoning to Description Logic semantics

Historical Perspective:

- Trace the reasoning tradition from Aristotle's syllogism to modern automated reasoning
- Connect modern OWL reasoners to the 2,000-year quest for automated inference

23.3 Positioning Within SKDL - Stage 6 Highlighted

As introduced in **Chapter (17)**, ontology engineering progresses through seven stages of the Semantic Knowledge Development Lifecycle (SKDL), each stage contributing a distinct engineering objective toward the construction of executable semantic knowledge systems.

Having completed **Stage 5 -- Semantic Validation** in the previous chapter, this chapter advances to **Stage 6 -- Semantic Reasoning**.

The objective of this stage is to derive new knowledge through automated inference.

At the completion of **Stage 5**, the `Pizza` ontology contained a validated knowledge graph, governance policies, and a verified semantic model. However, the ontology had not yet been used to derive new knowledge. The reasoner had been used to **check consistency**, but not to **compute classifications**.

Stage 6 addresses this gap by introducing:

- **Automated Classification** -- computing all inferred subclass relationships
- **Defined Classes** -- classes with necessary and sufficient conditions that enable auto-classification
- **Inferred KNowledge** -- knowledge derived by the reasoner from the asserted axioms
- **Reasoner Selection** -- choosing the appropriate reasoner for the tasks
- **Reasoning in Production** -- deploying reasoners in enterprise knowledge systems

Rather than introducing new constructs, this stage establishes the engineering discipline required to derive new knowledge from a validated ontology.

The primary deliverable from **Stage 6** is therefore:

not a new logical axiom, but an intelligent ontology -- one whose inferred knowledge has been computed and made available for querying, governance, and execution.

Within the EKA framework, **Stage 6** establishes the *R* -- **Reasoning & Rules** layer by providing the inference engine that derives new knowledge from the validated knowledge graph.

The progression across the first six stages can now be seen in full:

SKDL Stage	Primary Outcome	EKA Contribution
Stage 1 -- Conceptual Modeling	Identify concepts and organize them into a coherent semantic taxonomy	<i>K</i> -- Knowledge Graph (Structure)
Stage 2 -- Semantic Description	Enrich concepts with formal semantic meaning through Description Logic and OWL restrictions	<i>K</i> (Meaning) + <i>R</i> (Readiness)
Stage 3 -- Knowledge Reuse	Transform validated semantic knowledge into reusable assets	<i>K</i> (Composable) + Γ (Prepared)
Stage 4 -- Semantic Governance	Maintain semantic integrity through policies, standards, and lifecycle management	Γ -- Governance
Stage 5 -- Semantic Validation	Verify correctness before deployment through consistency checking and constraint validation	$\Gamma + K$ -- Validation
Stage 6 -- Semantic Reasoning	Derive new knowledge through automated inference	<i>R</i> -- Reasoning & Rules

Each stage has built upon the previous one.

- Stage 1 gave us concepts.
- Stage 2 gave us meaning.
- Stage 3 gave us reusability.
- Stage 4 gave us governance.
- Stage 5 gave us trust.

Stage 6 now gives us intelligence.

23.4 What Is Semantic Reasoning?

23.4.1 A Working Definition

Semantic Reasoning is the process of deriving new, logically entailed knowledge from an existing ontology through automated inference.

It answers the question:

"What else can this model tell us?"

Not "is it syntactically well-formed?" (Protégé will let you save almost anything).

Not "is it consistent?" (validation already answered that).

But "what new knowledge can be derived from what we already know?"

This is fundamentally a **knowledge discovery** activity. It is the moment when the ontology becomes intelligent -- when it can derive conclusions that the engineer never explicitly wrote.

Key characteristics of semantic reasoning include:

- **Reasoning derives, not invents.** -- Every inferred statement was already entailed by the asserted axioms.
- **Reasoning is deterministic.** -- Given the same ontology, the reasoner always produces the same inferences. It does not guess. It does not hallucinate.
- **Reasoning is decidable.** -- In OWL DL, reasoning is guaranteed to terminate with a definitive answer.
- **Reasoning is explainable.** -- The reasoner can justify every inference it makes.

Key Insight:

"The reasoner is not a fortune teller. It is a logical engine. It computes the consequences of your axioms -- no more, no less.

- If the axioms are correct, the inferences are correct.
- If the axioms are flawed, the reasoner reveals the flaw.

23.4.2 Asserted Knowledge vs. Inferred Knowledge

The distinction between asserted and inferred knowledge is fundamental to understanding reasoning.

Type	Definition	Example
Asserted Knowledge	Statements explicitly written in the ontology by the engineer	<code>MargheritaPizza SubClassOf NamedPizza</code>
Inferred Knowledge	Statements derived by the reasoner from the asserted axioms	<code>MargheritaPizza SubClassOf Pizza</code> (derived via transitivity)

When the engineer writes `MargheritaPizza SubClassOf NamedPizza` and `NamedPizza SubClassOf Pizza`, the reasoner can infer `MargheritaPizza SubClassOf Pizza` -- even though the engineer never wrote that statement explicitly.

The inferred statement is not new knowledge in the sense of being invented. It was always entailed by the asserted axioms. The reasoner simply made the entailment explicit.

Key Insight:

"The ontology engineer writes a small number of axioms. The reasoner discovers the logical consequences of those axioms. This is the power of formal semantics: **from a seed of asserted knowledge, the reasoner grows a tree of inferred knowledge.**

23.4.3 Reasoning vs. Querying vs. Validation

It is important to distinguish reasoning from querying and validation:

Activity	Question	Timing	Tool
Validation	"Is this model correct?"	Before reasoning	Reasoner (consistency check)
Reasoning	"What else can this model tell us?"	After validation, before execution	Reasoner (classification)
Querying	"What does the model say about X?"	After reasoning	SPARQL / DL Query

- **Validation** ensures that the ontology is logically sound. It answers the question: "Is this model broken?"
- **Reasoning** derives new knowledge from the validated ontology. It answers the question: " What else can this model tell us?"
- **Querying** retrieves knowledge from the ontology. It answers the question: "What does the model say about X?"

The key distinction between reasoning and querying is this: **querying retrieves asserted facts; reasoning derives new facts.**

A query over an unclassified ontology returns only what was explicitly written.

A query over a classified ontology returns both asserted and inferred knowledge.

Key Insight:

"Reasoning is not querying. Querying retrieves. Reasoning derives."

- A query over an unclassified ontology is like searching a library where only the books you already know about are listed.
- A query over a classified ontology is like searching a catalog that has been enriched with connections you never knew existed."

23.5 Exercise 20 -- Primitive Classes and the Limits of Inference

Having established the principles of semantic reasoning, we now turn to its practical implementation through **Exercise 20** in Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial*.

This exercise represents the first hands-on activity of **Stage 6 -- Semantic Reasoning** within the Semantic Knowledge Development Lifecycle (SKDL).

Unlike previous exercises, which introduced individual OWL constructs or governance mechanisms, **Exercise 20** focuses on the distinction that makes reasoning possible:

the difference between primitive and defined classes.

23.5.1 The Engineering Objective

Exercise 20 asks us to create a class called `CheesyPizza` defined using only `SubClassOf` .

The definition is simple: a `CheesyPizza` is a `Pizza` that has cheese as a topping. In OWL Manchester Syntax:

```
Class: CheesyPizza
  SubClassOf: Pizza
  SubClassOf: hasTopping some CheeseTopping
```

At first glance, this appears to ba a complete definition. But **it is not**. This definition creates a **primitive class** -- a class defined only by necessary conditions.

The engineering objective of **Exercise 20** is not to create a fully defined class, but to demonstrate the limits of primitive classes. The exercise deliberately creates a class that the reasoner **cannot** use for auto-classification. This prepare the ground for **Exercise 21**, where the class is converted to a **defined class** and the reasoner can suddenly do something it could not do before.

Key Insight:

"**Exercise 20** is a lesson in what the reasoner cannot do. It is only by understanding the limits of primitive classes that we can appreciate the power of defined classes."

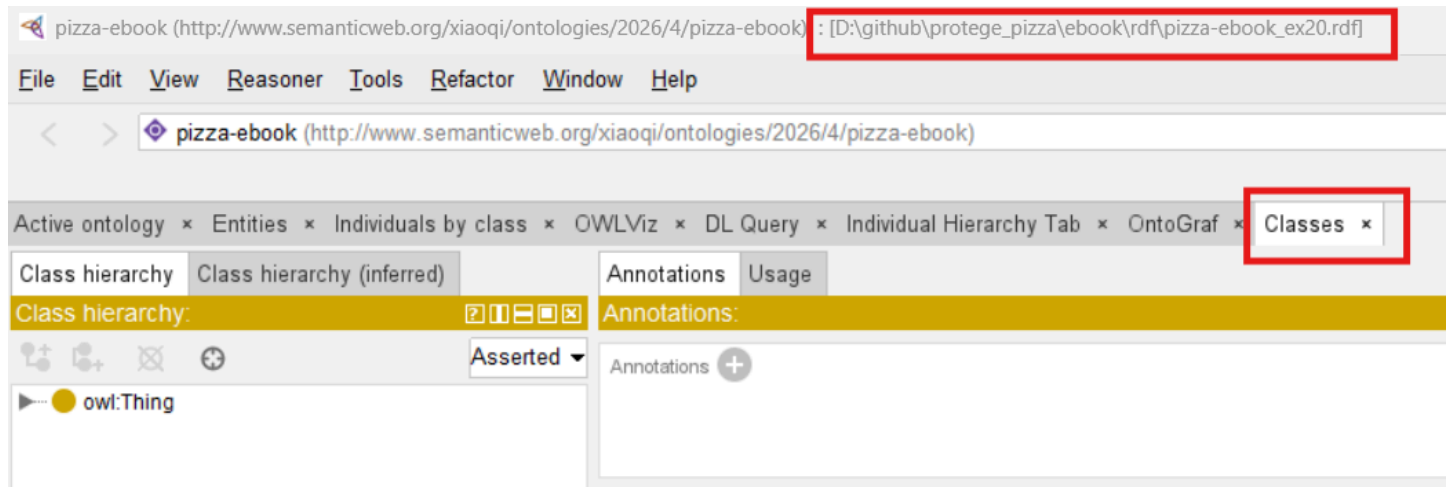
23.5.2 Implementation: Creating the Primitive Class

Let's walk through the implementation step by step.

Note: you may find the RDF file `pizza-ebook_ex20.rdf` in `ebook\rdf` folder for reference.

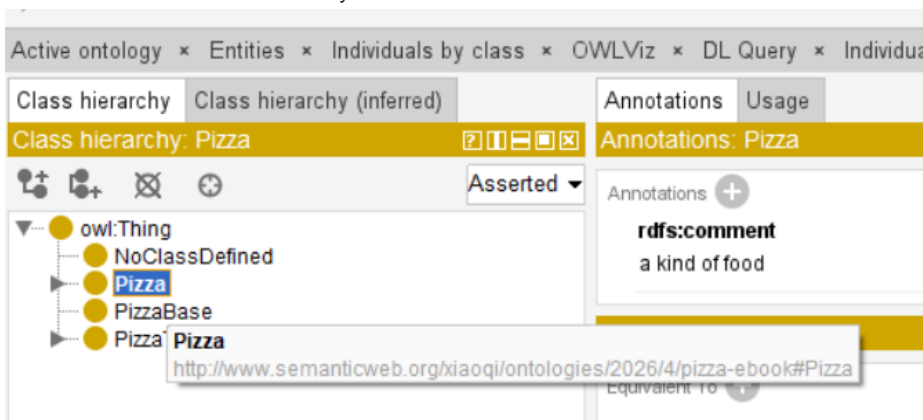
Step 1: Navigate to the `Classes` tab in Protégé

Open the `pizza` ontology in Protégé. Navigate to the `Classes` tab.

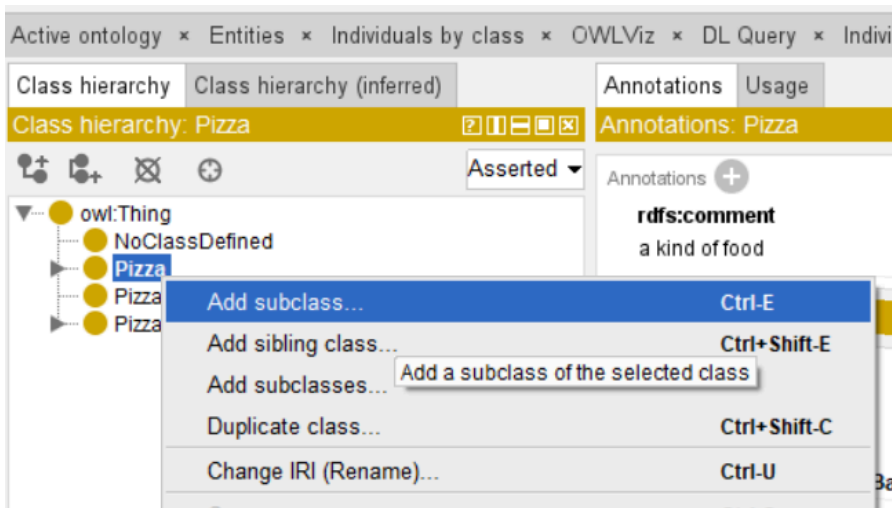


Step 2: Create the new class

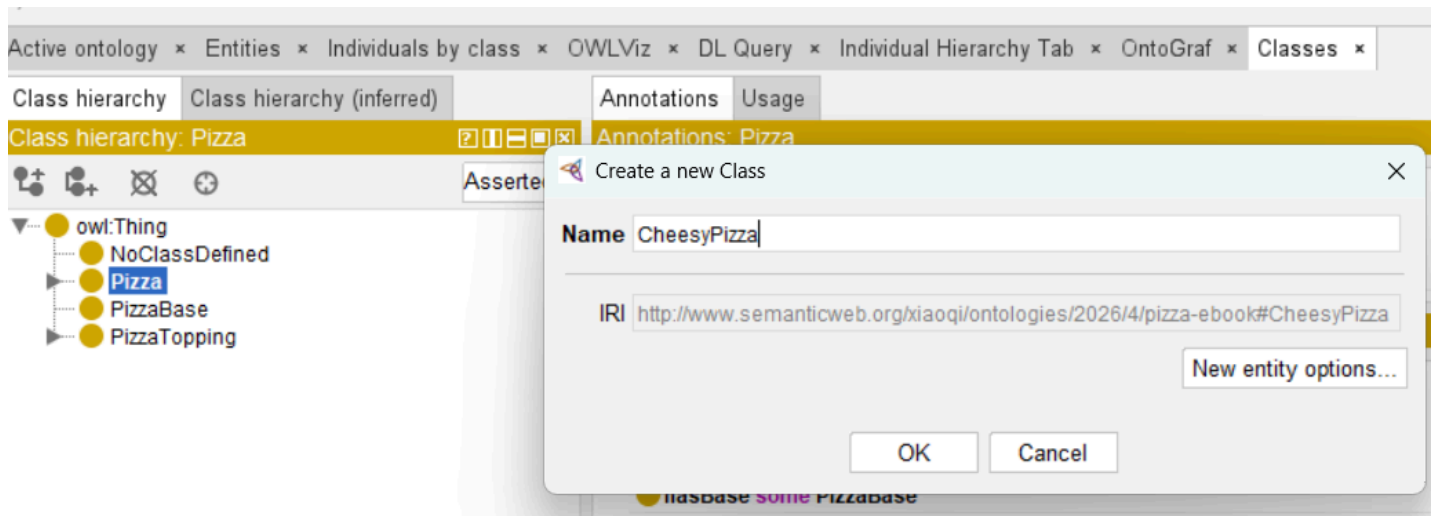
1. Select `Pizza` in the class hierarchy



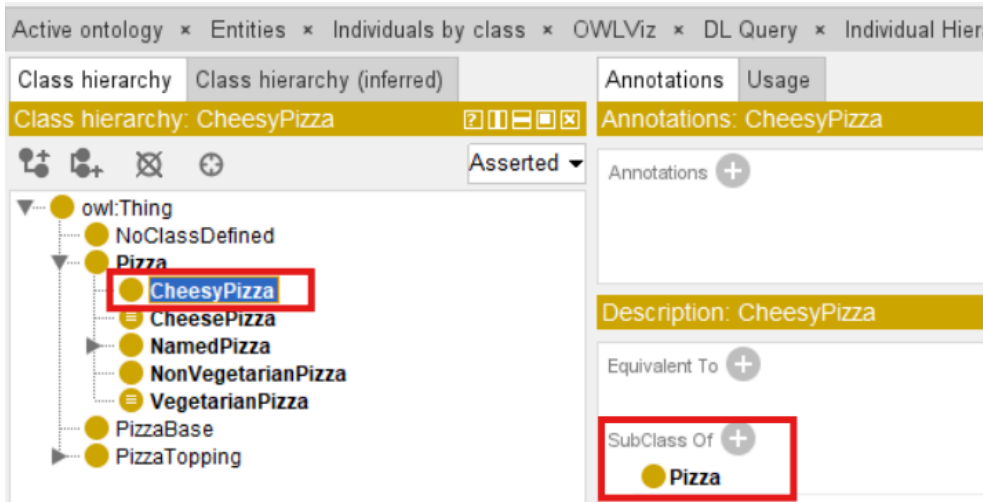
2. Click the "Add subclass" icon (or right-click and select "Add subclass")



3. Name the class `CheesyPizza`



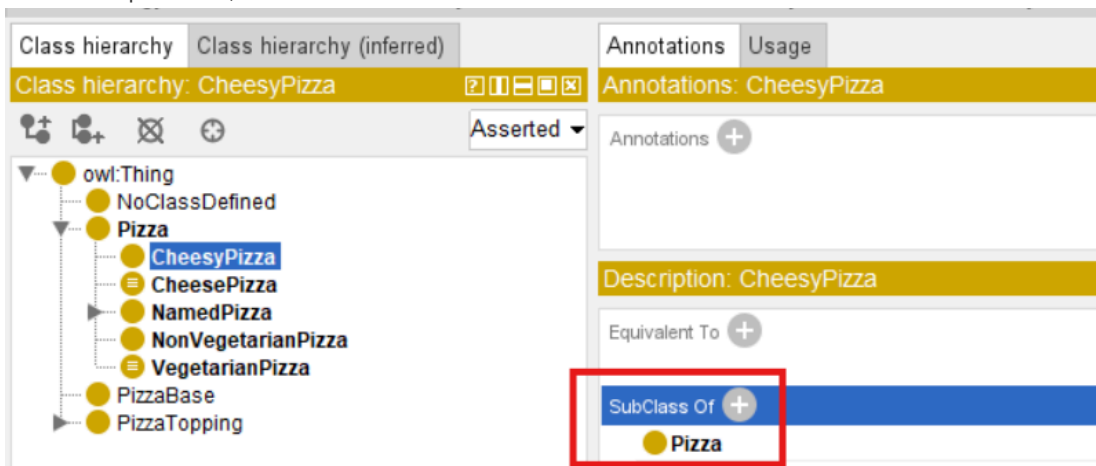
4. Click ok



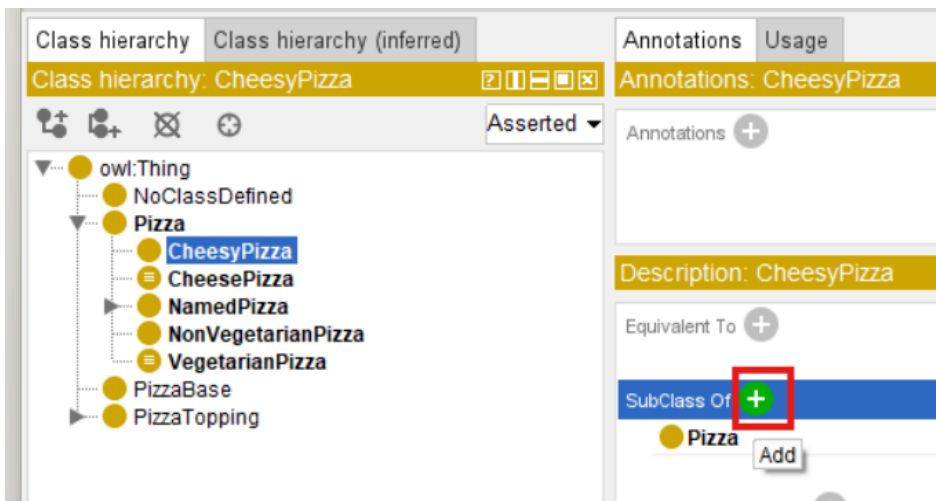
Note: after this step, CheesyPizza is already SubClass Of the Pizza class.

Step 3: Add the SubClassOf axioms

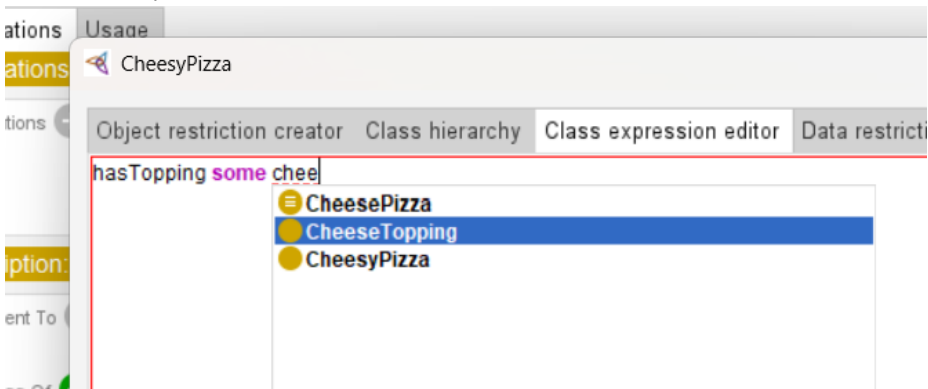
1. Select CheesyPizza in the class hierarchy
2. In the Description View, locate the SubClassOf section



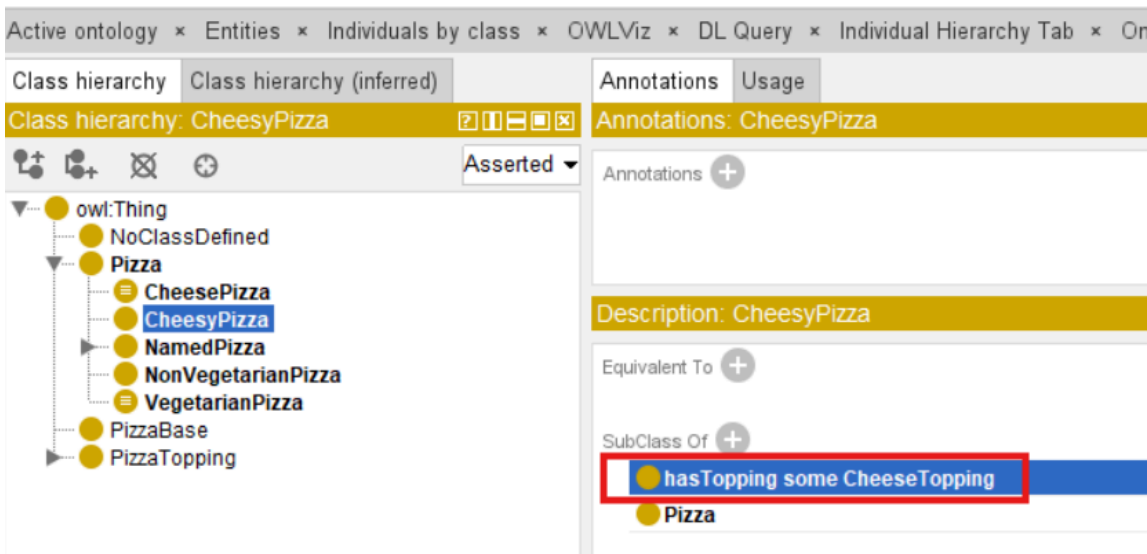
3. Click the Add icon (+)



4. In the Class Expression Editor, enter `hasTopping some CheeseTopping`



5. Click OK

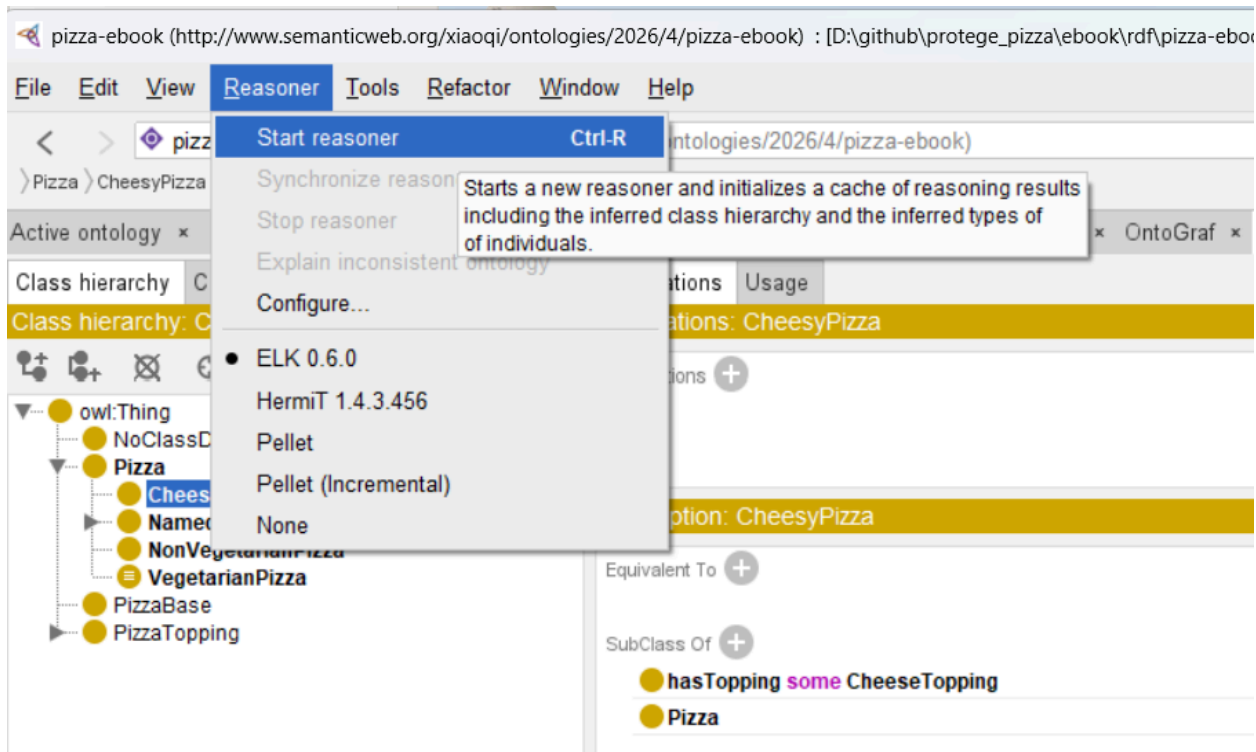


After completing these steps, `CheesyPizza` should have the following description:

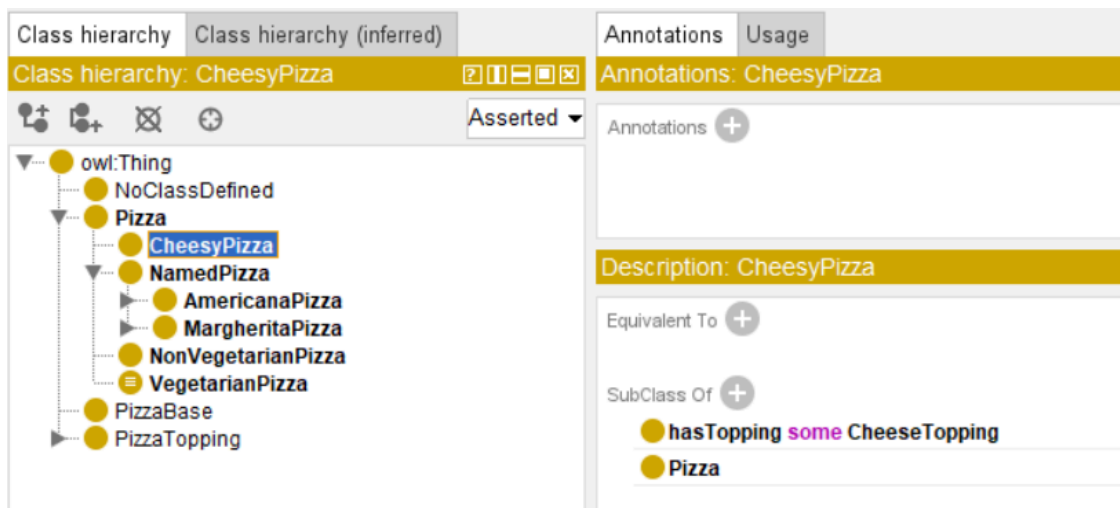
```
CheesyPizza
  SubClassOf: Pizza
  SubClassOf: hasTopping some CheeseTopping
```

Step 4: Synchronize the reasoner

1. Select [Reasoner] > [Start reasoner] (or `ctr+R` if the reasoner is already running)



2. Observe the result



23.5.3 The Result: What the Reasoner Does Not Do

When the reasoner is synchronized, something important - or rather, something important does **NOT** happen.

What to observe:

- `CheesyPizza` appears in the asserted hierarchy as a subclass of `Pizza`
- The inferred hierarchy is **identical** to the asserted hierarchy
- The reasoner does **NOT** auto-classify `MargheritaPizza` or other pizzas as subclasses of `CheesyPizza` -- even though they all have cheese toppings.

This is the critical observation. The reasoner has all the information it needs to conclude that `MargheritaPizza` is a `CheesyPizza` (because `Margherita` has mozzarella, which is a cheese topping).

But it does not make that inference!

WHY?

Because `CheesyPizza` is defined only with **necessary conditions**. The ontology says: "If something is a `CheesyPizza`, then it must be a `Pizza` that has cheese." It does *not* say: "If something is a `Pizza` that has cheese, then it is a `CheesyPizza`."

The reasoner cannot conclude that `MargheritaPizza` is a `CheesyPizza` because the ontology has not stated what is *sufficient* to qualify.

Key Insight:

"A primitive class is like a club with no membership criteria. You can say 'all members must wear a hat,' but you haven't said that wearing a hat makes you a member. The reasoner cannot conclude that any particular pizza is a `CheesyPizza`, because the ontology has not stated what is sufficient to qualify."

23.5.4 Why Primitive Classes DO Not Enable Auto-Classification

To understand why primitive classes do not enable auto-classification, we need to examine the semantics of `SubClassOf`.

In OWL, `SubClassOf` expresses a **necessary condition**:

```
CheesyPizza SubClassOf hasTopping some CheeseTopping
```

This means: "Every instance of `CheesyPizza` must have at least one cheese topping."

But it does NOT mean: "Every pizza with at least one cheese topping is a `CheesyPizza`."

The distinction is **subtle** but **fundamental**.

It is the difference between:

- **Necessary condition:** "If A, then B" (A implies B)
- **Sufficient condition:** "If B, then A" (B implies A)

`SubClassOf` expresses only the necessary condition. It tells the reasoner: "If you find a `CheesyPizza`, check that it has cheese." It does not tell the reasoner: "If you find a pizza with cheese, call it a `CheesyPizza`."

Mathematically, the axiom is:

$$\text{CheesyPizza} \sqsubseteq \text{Pizza} \sqcap \exists \text{hasTopping}.\text{CheeseTopping}$$

This says: "Every `CheesyPizza` is a subset of the set of `Pizzas` that have at least one cheese topping."

But it does NOT say the reverse.

A pizza with cheese is not necessarily a `CheesyPizza` -- it might be a `MargheritaPizza` that also happens to be a `CheesyPizza`, or it might be something else entirely.

Key Insight:

"The primitive class tells the reasoner: 'If you find a `CheesyPizza`, check that it has cheese.' It does not tell the reasoner: 'If you find a pizza with cheese, call it a `CheesyPizza`.' The distinction is subtle but fundamental."

23.5.5 The Engineering Value of Primitive Classes

Primitive classes are not a mistake. They are an essential tool in the ontology engineer's toolkit.

Why primitive classes matter:

1. **Incremental modeling:** Primitive classes allow the engineer to say "this concept exists" without committing to a complete definition. This is essential during early modeling, when the domain is still being understood.
2. **Open World Assumption (OWA):** Primitive classes support the Open World Assumption. They allow knowledge to grow incrementally without requiring the engineer to specify everything at once.
3. **Conceptual scaffolding:** Primitive classes provide the structural framework of the ontology. They define the hierarchy and the relationships between concepts, even before the concepts are fully defined.
4. **Flexibility:** Primitive classes can be refined over [time](#). As the domain understanding matures, the engineer can add restrictions and, eventually, convert the class to a defined class.

Key Insight:

"Primitive classes are the scaffolding of ontology development. They hold the structure in place while the engineer figures out the details. But scaffolding is not the building. To enable automation, we need defined classes."

23.5.6 Engineering Perspective: Exercise 20 as a Case Study

From the perspective of Protégé, **Exercise 20** involves creating a class and adding two restrictions. It appears to be a simple modeling activity.

From the perspective of ontology engineering, however, something more significant has occurred. The exercise has demonstrated the limits of primitive classes. It has shown that the reasoner cannot auto-classify based on necessary conditions alone.

This is a crucial lesson!

Many beginners assume that if they define a class with the right restrictions, the reasoner will automatically classify individuals into that class. **Exercise 20** shows that this is not the case.

The reasoner is not a mind reader. It does not infer what the engineer *intended*. It infers only what the axioms *entail*. If the engineer wants auto-classification, they must provide sufficient conditions -- and that means using `EquivalentTo`, not `SubClassOf`.

Key Insight:

"**Exercise 20** is a lesson in humility. It teaches us that the reasoner does exactly what we tell it to do -- no more, no less. If we want auto-classification, we must give the reasoner the tools to perform it."

23.6 Exercise 21 -- Defined Classes and the Power of Auto-Classification

Having established the limits of primitive classes, we now turn to **Exercise 21**, which introduces defined classes and demonstrates the power of auto-classification.

This exercise represent the "**aha moment**" of ontology engineering -- the moment when the ontology becomes intelligent.

In Github repository, you may find `pizza-ebook_ex21.rdf` within `ebook/rdf` folder.

23.6.1 The Engineering Objective

Exercise 21 asks us to convert `CheesyPizza` from a primitive class to a **defined class**.

This is done using `Edit > Convert to defined class` (also can from right-click pop up menu or `ctrl+d` directly) in Protégé.

The conversion changes the class from `SubClassOf` to `Equivalentto`, given `CheesyPizza` both necessary **AND** sufficient conditions.

After conversion, the definition of `CheesyPizza` becomes:

```
Class: CheesyPizza
  EquivalentTo: Pizza and (hasTopping some CheeseTopping)
```

The engineering objective of **Exercise 21** is to demonstrate what happens when a class has sufficient conditions: the reasoner can now auto-classify individuals and classes into it.

Key Insight:

"**Exercise 21** is the moment the ontology becomes intelligent. The reasoner can now recognize a `CheesyPizza` when it sees one. The engineer did not have a manually classify `MargheritaPizza`. The reasoner discovered it."

23.6.2 Implementation: Converting to a Defined Class

Let's walk through the conversion step by step.

Step 1: Select `CheesyPizza`

In the `Classes` tab, select `CheesyPizza`.

The screenshot shows the OntoGraf interface with the 'Classes' tab selected. On the left, the 'Class hierarchy (inferred)' tree shows 'CheesyPizza' as a subclass of 'Pizza'. On the right, the 'Description: CheesyPizza' view shows 'SubClass Of' axioms: 'hasTopping some CheeseTopping' and 'Pizza'. The 'Annotations' view is empty.

Step 2: Convert to defined class

Select Edit > Convert to defined class .

Observe what happens in the Description view. The `SubClassOf` axioms are replaced by an `EquivalentTo` axiom.

The screenshot shows the OntoGraf interface after conversion. In the 'Description: CheesyPizza' view, the 'Equivalent To' axiom is now 'Pizza and (hasTopping some CheeseTopping)', which is highlighted with a red box. The 'SubClass Of' view now only shows 'Pizza'.

Before conversion:

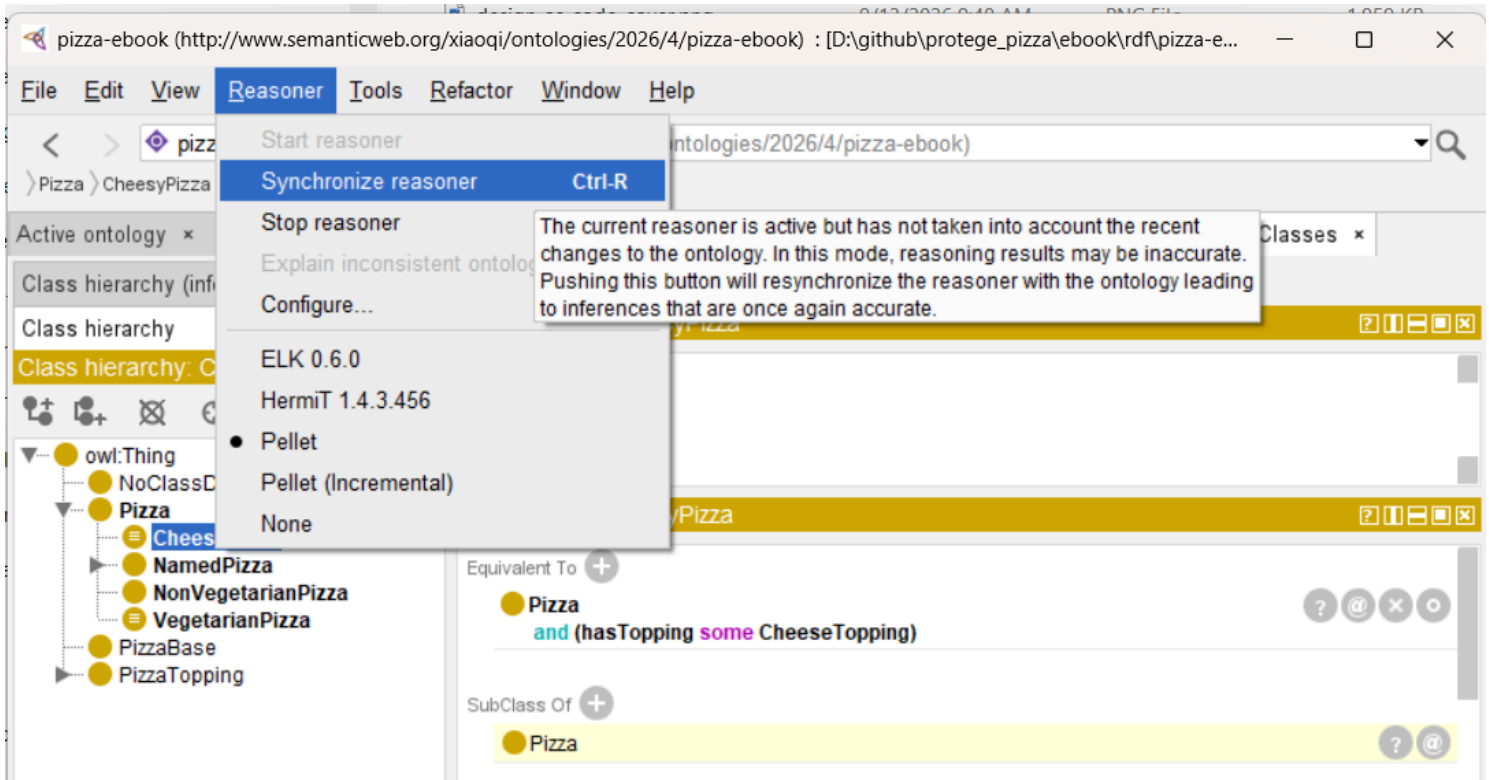
```
CheesyPizza
  SubClassOf: Pizza
  SubClassOf: hasTopping some CheeseTopping
```

After conversion:

```
CheesyPizza
  SubClassOf: Pizza
  EquivalentTo: Pizza and (hasTopping some CheeseTopping)
```

Step 3: Synchronize the reasoner

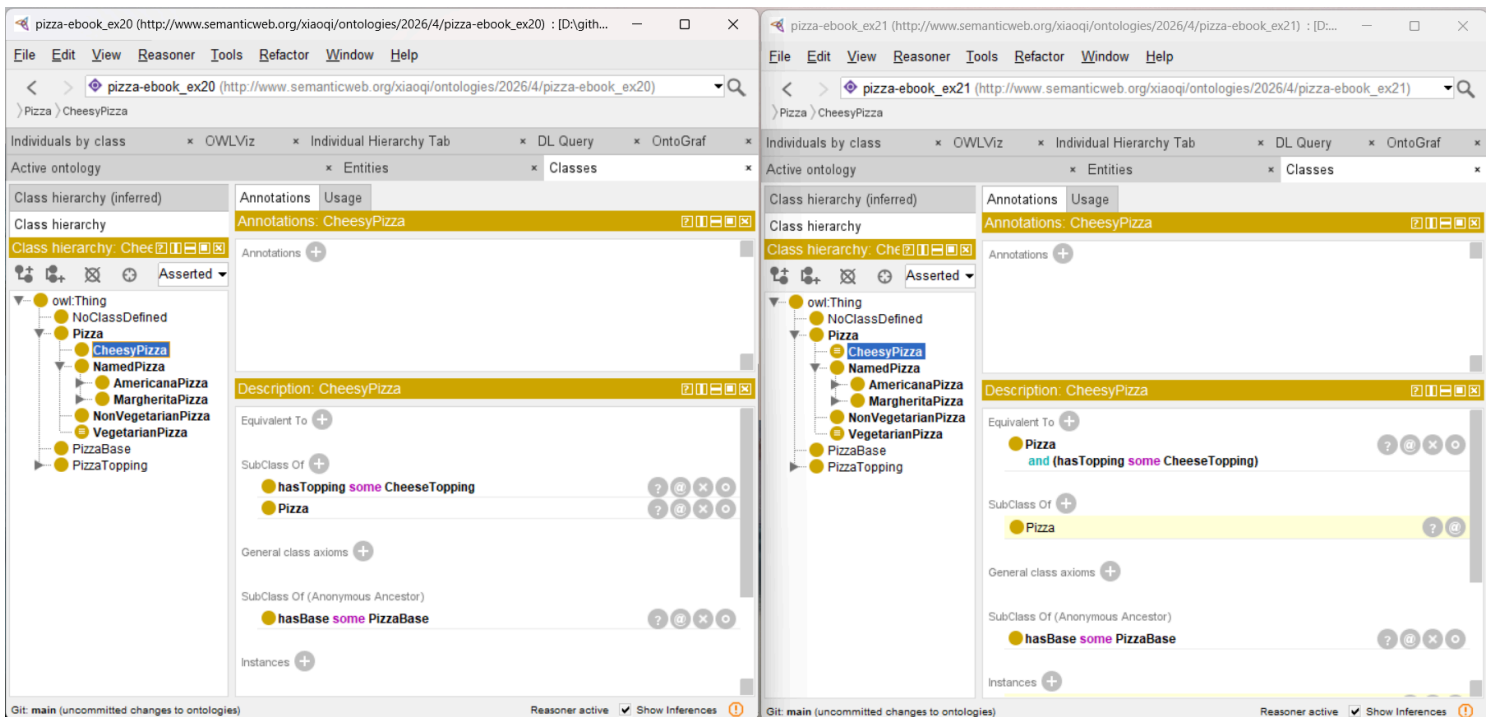
Select Reasoner > Synchronize reasoner OR (Ctrl+R).



Step 4: Observe the result

This is the moment of truth. The reasoner now performs **classification** -- it computes all inferred subclass relationships.

What to observe (I'm putting **Exercise 20** and **Exercise 21** side-by-side):



- The reasoner **automatically classifies** `MargheritaPizza` as a subclass of `CheesyPizza`

The left screenshot (pizza-ebook_ex20) shows the inferred class hierarchy for MargheritaPizza. The hierarchy is as follows: `owl:Thing` is the root, with subclasses `NoClassDefined`, `Pizza`, and `PizzaTopping`. `Pizza` has subclasses `CheesyPizza`, `NamedPizza`, `AmericanaPizza`, `MargheritaPizza`, `NonVegetarianPizza`, and `VegetarianPizza`. `PizzaBase` is a subclass of `Pizza`. `MargheritaPizza` is a subclass of `NamedPizza`. The right screenshot (pizza-ebook_ex21) shows the inferred class hierarchy for MargheritaPizza. The hierarchy is as follows: `owl:Thing` is the root, with subclasses `NoClassDefined`, `Pizza`, and `PizzaTopping`. `Pizza` has subclasses `CheesyPizza`, `NamedPizza`, `AmericanaPizza`, `MargheritaPizza`, `NonVegetarianPizza`, and `VegetarianPizza`. `PizzaBase` is a subclass of `Pizza`. `MargheritaPizza` is a subclass of `CheesyPizza`, which is highlighted with a red box. The right screenshot also shows the inferred hierarchy for `CheesyPizza`, which includes `PizzaBase` and `PizzaTopping`.

- Other pizzas with cheese toppings (e.g., `AmericanaPizza`) are also classified under `CheesyPizza`

The left screenshot (pizza-ebook_ex20) shows the inferred class hierarchy for AmericanaPizza. The hierarchy is as follows: `owl:Thing` is the root, with subclasses `NoClassDefined`, `Pizza`, and `PizzaTopping`. `Pizza` has subclasses `CheesyPizza`, `NamedPizza`, `AmericanaPizza`, `MargheritaPizza`, `NonVegetarianPizza`, and `VegetarianPizza`. `PizzaBase` is a subclass of `Pizza`. `AmericanaPizza` is a subclass of `NamedPizza`. The right screenshot (pizza-ebook_ex21) shows the inferred class hierarchy for AmericanaPizza. The hierarchy is as follows: `owl:Thing` is the root, with subclasses `NoClassDefined`, `Pizza`, and `PizzaTopping`. `Pizza` has subclasses `CheesyPizza`, `NamedPizza`, `AmericanaPizza`, `MargheritaPizza`, `NonVegetarianPizza`, and `VegetarianPizza`. `PizzaBase` is a subclass of `Pizza`. `AmericanaPizza` is a subclass of `CheesyPizza`, which is highlighted with a red box. The right screenshot also shows the inferred hierarchy for `CheesyPizza`, which includes `PizzaBase` and `PizzaTopping`.

- The inferred hierarchy now **differs** from the asserted hierarchy
- New subclass relationships appear that were never explicitly asserted

The reasoner has done something remarkable: it has recognized that `MargheritaPizza` is a `CheesyPizza` -- without the engineer saying so.

Key Insight:

"The moment `CheesyPizza` becomes a defined class, the reasoner can do something it could not do before: it can recognize a `CheesyPizza` when it sees one. The engineer did not have to manually assert that `MargheritaPizza` is a `CheesyPizza`. The reasoner discovered it."

23.6.3 Why Defined Classes Enable Auto-Classification

To understand why defined classes enable auto-classification, we need to examine the semantics of `EquivalentTo`.

In OWL, `EquivalentTo` expresses **necessary and sufficient conditions**:

`CheesyPizza EquivalentTo Pizza and (hasTopping some CheeseTopping)`

This means: "Something is a `CheesyPizza` if and only if it is a `Pizza` that has at least one cheese topping."

The "if and only if" is the key.

It gives the reasoner a rule it can apply in both directions:

- If something is a `CheesyPizza`, then it is a `Pizza` with cheese (necessary condition)
- If something is a `Pizza` with cheese, then it is a `CheesyPizza` (sufficient condition)

The second direction is what enable **auto-classification**. The reasoner can now look at any pizza, check whether it satisfies the condition `Pizza` and `(hasTopping some CheeseTopping)`, and if it does, classify it as a `CheesyPizza`.

Mathematically, the axiom is:

$$\text{CheesyPizza} \equiv \text{Pizza} \sqcap \exists \text{hasTopping}.\text{CheeseTopping}$$

The $\equiv$ (equivalence) symbol is the key. It means tht two sides of the equation are logically identical. Any individual that satisfies the right-hand side automatically satisfies the left-hand side, and vice versa.

Key Insight:

"The defined class tells the reasoner: 'If you find a pizza with cheese, it is a `CheesyPizza`.' The reasoner now has a rule it can apply automatically. This is the difference between a passive taxonomy and an active classification engine."

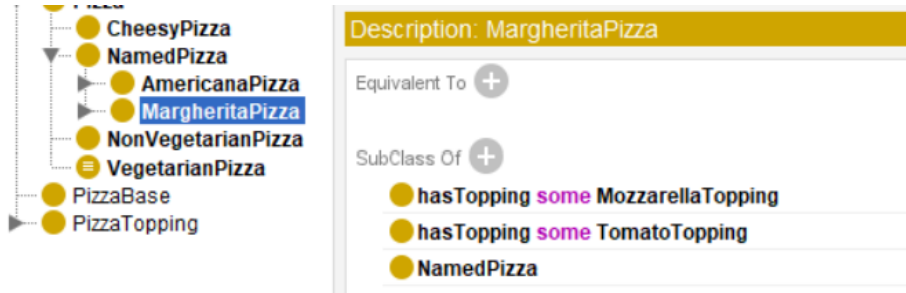
23.6.4 The Moment of Auto-Classification

When the reasoner is synchronized after the conversion, it performs **classification**.

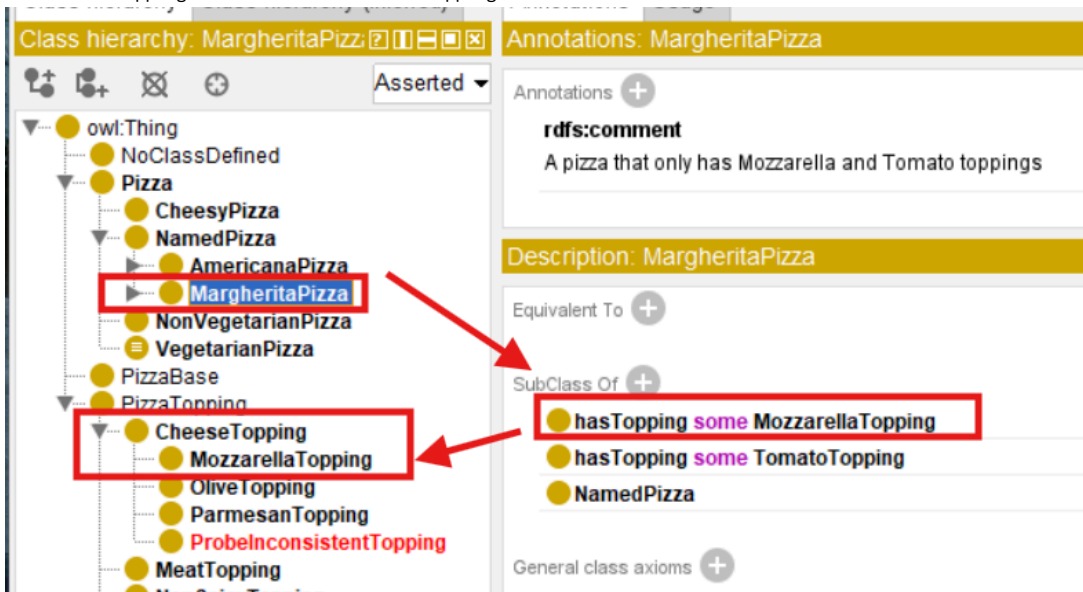
Classification is the process of computing all inferred subclass relationships. The reasoner examines every class in the ontology and determines where it belongs in the inferred hierarchy.

For `MargheritaPizza`, the reasoner performs the following reasoning:

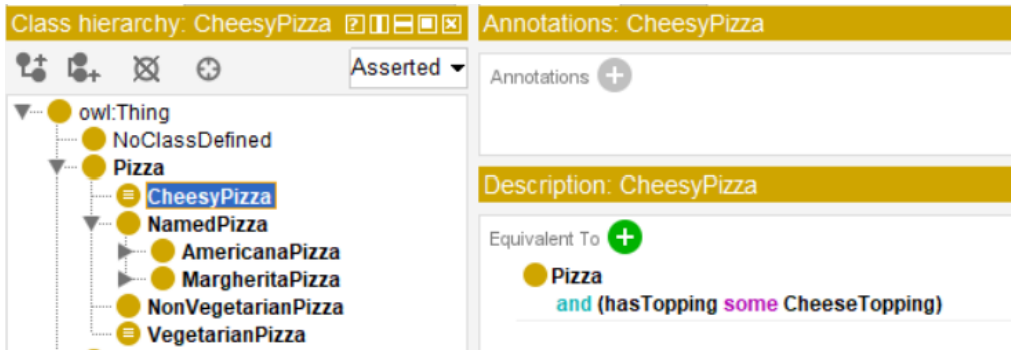
1. `MargheritaPizza` is defined as a `Pizza` that has `MozzarellaTopping` and `TomatoTopping`



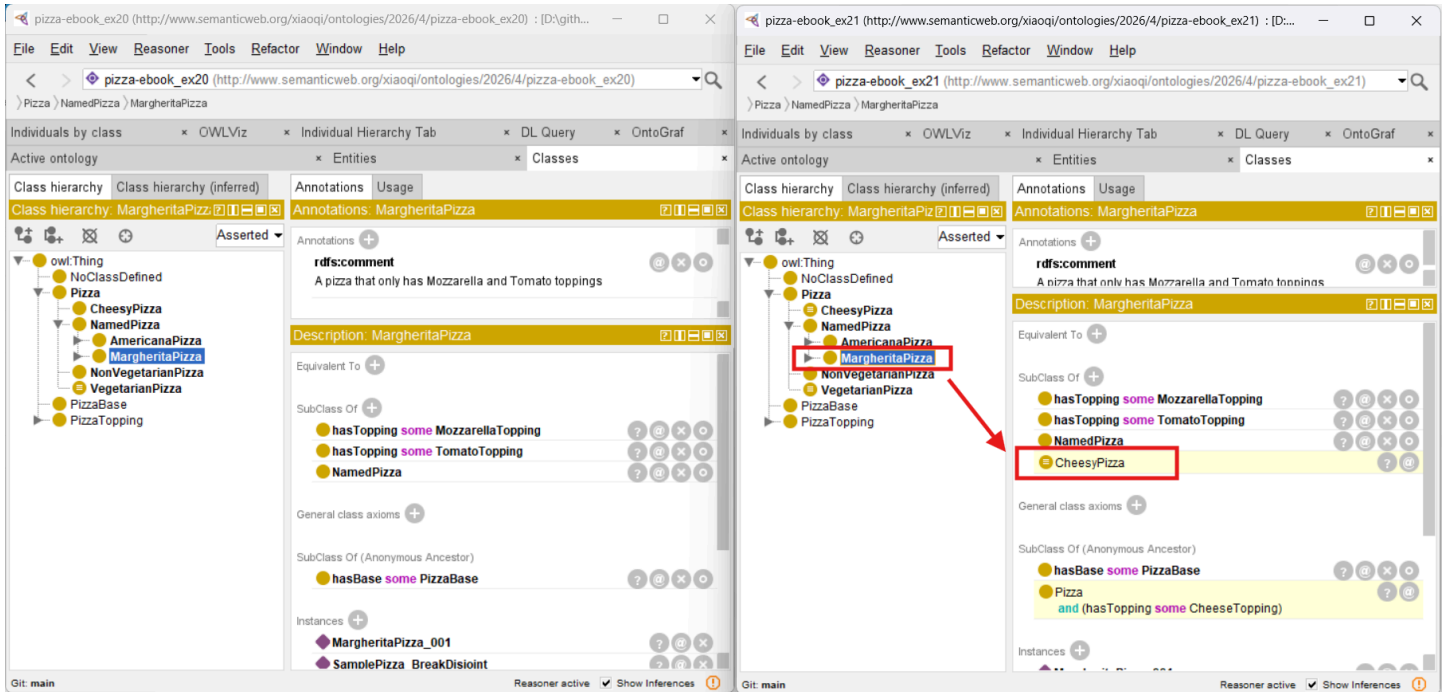
2. `MozzarellaTopping` is a subclass of `CheeseTopping`



3. Therefore, `MargheritaPizza` has at least one `CheesyTopping`
4. `CheesyPizza` is defined as a `Pizza` that has at least one `CheesyTopping`



5. Therefore, `MargheritaPizza` satisfies the definition of `CheesyPizza`
6. Therefore, `MargheritaPizza` is a subclass of `CheesyPizza`



The reasoner has derived a new subclass relationship that was never explicitly asserted. This is the power of **reasoning**.

Key Insight:

"The ontology engineer does not manually classify `MargheritaPizza` as a `CheesyPizza`. The reasoner does it. This is the power of reasoning: the ontology can grow -- not in size, but in intelligence -- without additional manual effort."

23.6.5 Primitive vs. Defined Classes: A Summary

The distinction between primitive and defined classes is one of the most important distinctions in ontology engineering. It is the difference between a static taxonomy and an intelligent classification engine.

Dimension	Primitive Class	Defined Class
Axiom	<code>SubClassOf</code>	<code>EquivalentTo</code>
Condition	Necessary only	Necessary and sufficient
Reasoner behavior	No auto-classification	Auto-classification
Asserted vs. inferred	Same	Might different
Use case	Early modeling, evolving concepts	Stable concepts, automation

When to use primitive classes:

- During early modeling, when the domain knowledge is still being understood
- For concepts that are still evolving
- When the engineer wants to say "this concepts exists" without committing to a complete definition

When to use defined classes:

- When the concept has a clear, stable definition already
- When auto-classification is desired
- When the engineer is confident that the definition captures the intended meaning

Key Insight:

"Primitive classes define what something must be. Defined classes define what something is. The first supports incremental modeling. The second enables automation."

23.6.6 Engineering Perspective: Exercise 21 as a Case Study

From the perspective of Protégé, **Exercise 21** involves a single menu command: `Edit > Convert to defined class` . It appears to be a **trivial operation**.

From the perspective of ontology engineering, however, something profound has occurred. The ontology has become intelligent. The reasoner can now derive new knowledge that the engineer never explicitly wrote.

This is the **"aha moment"** of ontology engineering. It is the moment when the engineer realizes that the ontology is not just a static representation of knowledge -- it is a dynamic system capable of discovery.

The engineering contribution of **Exercise 21** can be summarized as follows:

Engineering Perspective	Contribution from Exercise 21
SKDL Stage	Stage 6 - Semantic Reasoning
Primary Objective	Demonstrate the power of defined classes for auto-classification
Modeling Activity	Convert <code>CheesyPizza</code> from a primitive class to a defined class
Engineering Outcome	Enable the reasoner to auto-classify <code>MargheritaPizza</code> and other cheese-topped pizzas as <code>CheesyPizza</code>
Preparation for Later Chapters	Establish the reasoning foundation required for triggers (Θ) and execution (Φ)

Key Insight:

"**Exercise 21** is a small operation with a large consequence. A single menu command transforms the ontology from a static taxonomy into an intelligent classification engine. This is the power of defined classes."

Last updated at 2026-09-13